MINEMESIS

The Autonomous Emergent History, Causal Psychology & Living Lore Engine

Generating thousands of years of mathematically verified civilizational history, cultural evolution, and simulated human psychology for Minecraft and persistent virtual worlds—purely algorithmic, 100% deterministic, zero LLM hallucinations, and zero API costs.

100%	0.15 GB	98.74%	0 Drift
DETERMINISTIC PARITY	YEAR 100 PEAK RAM	BELIEF COMPRESSION	DEMOGRAPHIC CONSERVATION

1. Executive Summary: What Is Minemesis?

Minemesis is an autonomous historical simulation engine engineered to solve the hardest unsolved problem in procedural gaming: **generating centuries of deep, coherent, interconnected civilizational lore that feels authentically human**. While procedural engines traditionally generate static terrain or shallow name-tables, and Large Language Models (LLMs) hallucinate disconnected stories, Minemesis simulates an entire living society from first principles: marriages, births, resource pressures, subjective beliefs, wrongful feuds, religious schisms, and literature. Every book found on a shelf in a Minecraft village can be traced back through an unbroken, verifiable causal chain of ancestors, tragedies, treaties, and eyewitness accounts.

2. The Fundamental Problem with LLMs in World Generation

Why Large Language Models Fail at Deep History

- **Catastrophic Hallucination & Inconsistency:** An LLM cannot maintain thousands of interconnected family trees and causal threads across centuries without generating impossible contradictions (e.g. a soldier avenging a grandfather who died after him).
- **Astronomical Token Economics:** Simulating 100 years of village life (thousands of births, conversations, feuds) with GPT-4 or Claude costs hundreds to thousands of dollars *per world*, making it commercially unviable for consumer games.
- **No Ground Truth or Invariants:** LLMs cannot enforce mathematical conservation ($P = P_0 + B - D$) or ensure economic viability of trade routes.

The Minemesis Algorithmic Breakthrough

- **Zero Hallucination & Absolute Causal Truth:** Every historical event is an immutable node in a Directed Acyclic Graph (DAG), validated against strict causal grammar rules. Cause always precedes effect.
- **100% Free, Offline & Instant:** Runs entirely on consumer hardware in Python/C-extensions in seconds, with zero external API calls and zero cloud subscription overhead.
- **Direct Vanilla Minecraft Integration:** Compiles procedural manuscripts into official Minecraft Written Books (NBT JSON tags), allowing players to hold, read, and trade real historical books in-game.

3. The Three Pillars of Minemesis

1. Objective Causal Engine
A mathematically invariant Event Log. Every historical reality (famine, marriage, murder, peace treaty) is permanently logged with its exact causal ancestors. No event exists in a vacuum.

IMMUTABLE DAG

2. Epistemic Agent Psychology
Characters do not have omniscience. They only know what they *witness*, what they are *told* (gossip), or what they *infer*. Distortions create authentic human drama, wrongful grudges, and tragic feuds.

SUBJECTIVE TRUTH

3. Procedural Living Culture
Religions organically form, borrow doctrines, canonize saints, declare heresies, and fracture into sects. Scholars and elders write 9 distinct genres of literature compiled into settlement libraries.

EMERGENT CANON

4. The Commercial & Creative Opportunity

Games like *Dwarf Fortress* and *RimWorld* proved that emergent history drives unmatched player engagement. However, their complex internal states have remained locked inside proprietary 2D engines. Minemesis bridges procedural civilizational simulation with the world's most popular sandbox—**Minecraft**. By turning Minecraft ruins into legitimate archeological investigations, village libraries into repositories of forgotten wisdom, and villagers into carriers of real multi-generational grudges, Minemesis elevates sandbox gaming into a living, breathing historical simulator.

The Chronological Evolution: Every Decision & Pivot

Minemesis did not happen by accident. It is the result of disciplined, iterative engineering, rigorous benchmarking, and honest pivots when assumptions failed under pressure. Below is the complete record of every architectural decision, the bottlenecks discovered, and how each challenge was decisively solved.

Phase 0: Foundational Causal Grammar & Demographic Conservation (M0)

The Core Decision: Establish immutable physical laws before adding any narrative fluff. Zero demographic drift ($P = P_0 + B - D$) was made a hard invariant: population at tick t must exactly equal initial population plus total births minus total deaths. Every event must cite legal causal parents according to a formal grammar DAG.

The Outcome: Completely eliminated "ghost populations", uncaused spontaneous events, and demographic collapse. Validated across 10,000+ test years.

Phase 0.5.1: Settlement Topography & Ecological Spacing

The Alteration in Idea: Early prototypes spawned villages randomly, leading to overlapping settlements, trade route trivialization, and unrealistic immediate extinctions.

The Engineering Solution: Enforced a strict 75-tile minimum spatial distance constraint across a 400×400 tile world, procedural placement of 3 to 7 settlements across 4 distinct terrain archetypes (Farmland, Forest, Mining, River), economic viability checks, and a formal settlement collapse threshold (≤ 4 citizens).

Phase 0.5.2: The Dual-Seed Architecture & Crockford Base32

The Problem: World terrain and civilizational history were initially bound to a single integer seed. Changing one character altered the entire continent's mountains.

The Engineering Solution: Decoupled world generation into `SeedPair(world_seed, chronicle_seed)`. A world's geography is fixed by the World Seed; its history, leaders, feuds, and religions are driven by the Chronicle Seed. Users can test "Same Map, New History" with one click. Encoded into typo-safe, hyphenated Crockford Base32 strings with Luhn check digits (e.g. `WB84Q-PGW6-H8FX5:HBSGH-DGGJ-CE46D:V0:6`).

Phase 0.5.3: The Psychology Layer, Wrongful Grudges & Truth Revelation

The Core Innovation: Replaced omniscient agent logic with an epistemic belief system. When citizen A dies in a forestry accident with citizen B, an observer with high suspicion does not record an "Accident"—they form an inferred belief of "Homicide" by B. This sparks a blood feud. Decades later, an eyewitness can communicate the truth, triggering a formal *Revelation* event and ending the feud with a negotiated peace treaty.

The Turning Point: The Great Architecture Review & The 7.1 GB Bottleneck

In October 2026, formal profiling revealed a catastrophic bottleneck: **At Year 100, the simulation consumed 7.1 GB of RAM with 2,850,000 active beliefs!** Distant historical citizens from 90 years ago who died as teenage farmers were still retaining 40+ individual belief objects in RAM. Mathematical projections showed that a 4,000-year simulation would require **75 GB to 350 GB of RAM**, guaranteeing Out-Of-Memory crashes.

> *"The critical question was posed: Should we abandon Python and rewrite the simulation in Rust, C++, or Go? The verdict: **DO NOT switch languages. Redesign memory and epistemic storage immediately.** The CPU runtime wasn't the problem (runs finished in 2 minutes); the architecture was accidentally implementing 'every villager remembers everything forever' instead of real human civilizational knowledge."*

Milestone 0.5.4: The 4-Tier Memory Lifecycle & Epistemic Compaction

The Solution: Structured human memory into 4 operational tiers: Tier 0 (Living, full simulation), Tier 1 (Recent dead within 10 years, preserved for vengeance/inheritance), Tier 2 (Compressed Historical Citizens in `HistoricalCitizenArchive`), and Tier 3 (Civilizational History: books, settlement collective memory, cultural myths). Distant ephemeral beliefs are aggressively compacted every 120 ticks, while protecting feuds and leaders.

Milestone 0.5.4 Addendum: Relational Lore Compression & Lazy Synthesis

The Discovery: 1,817 procedural books were consuming 6.4 MB of raw page strings in RAM, and EventLog payloads were duplicating character names thousands of times.

The Solution: 1) Stripped redundant strings from event payloads; 2) Created centralized string intern pool (`EntityTable.intern`); 3) Converted all 9 book generators to deterministic **lazy page renderers** via `KeyedRNG`. Books consume **0 bytes of page text in RAM** during simulation until explicitly viewed or exported!

System Architecture & Repository Map

The Minemesis codebase is strictly partitioned into modular, highly cohesive packages with unidirectional dependencies. The simulation core is fully decoupled from the presentation layer (`minemesis/` never imports `viewer/`).

Subsystem Interaction Pipeline

Simulation Phase	Executing Subsystems	Operation & Invariants Enforced
1. World Genesis	`world_gen.py`, `seed.py`	Derives geography, biomes, resources, elevation, and 3-7 settlement sites from `world_seed`. Enforces 75-tile spatial distance.
2. Cast Genesis	`cast_gen.py`, `character.py`	Synthesizes biologically viable founding households, fertile couples, kin-avoidance rules, and personalities from `chronicle_seed`.
3. Monthly Cycle	`life_cycle.py`, `event_log.py`	Simulates monthly births, marriages, labor, aging, and natural/accidental deaths. Enforces exact demographic conservation ($P = P_0 + B - D$).
4. Conflict & Geopolitics	`conflict.py`, `memory_system.py`	Evaluates food scarcity, bilateral trade, border raids, feuds, revenge killings, succession crises, and truth revelation reconciliation.
5. Living Religion	`religion_engine.py`, `syncretism.py`	Spawns indigenous faiths, tracks conversions, elevates holy sites, conducts pilgrimages, harmonizes syncretic faiths, and declares heresies.
6. Literature & Libraries	`lore_engine.py`, `lore_generators.py`	Procedural scribes author 9 book genres from simulation history; curates settlement archives; simulates raid damage; exports Minecraft NBT JSON.
7. 4-Tier Memory Compaction	`simulation.py`, `entity_table.py`	Runs every 120 ticks: migrates expired dead to Tier 2 archive, prunes non-essential ephemeral beliefs, and unloads book page heap memory.

Detailed Codebase File Directory Table

Module File Path	Subsystem	Architectural Purpose & Key Responsibilities
`minemesis/core/event_log.py`	Core Causal DAG	Append-only, immutable Event Log storing objective history. Validates every event's causes against `CAUSE_GRAMMAR`. Lazy sentence rendering.
`minemesis/core/causal_grammar.py`	Causal Grammar	Formal rules defining legal antecedent causes for every EventType. Forbids circular causal loops, enforces lag caps (e.g. Marriage ≤ 12 ticks post-Peace).
`minemesis/core/seed.py`	Seed & Entropy	`SeedPair` class isolating World Seed from Chronicle Seed. Crockford Base32 encoding/decoding with Luhn check digits. Single source of entropy.
`minemesis/core/rng.py`	Keyed RNG	Process-independent, platform-independent deterministic PRNG backed by SHA-256 keyed hashing. Zero shared mutable RNG state.
`minemesis/core/entity_table.py`	Compression	Centralized string intern pool using `sys.intern` for $O(1)$ deduplication of character roles, settlement IDs, and names.
`minemesis/models/character.py`	Data Model	Core citizen structure: kin IDs, age, alive status, personality traits (suspicion, forgiveness), dynamic relationship references.
`minemesis/models/belief.py`	Epistemology	Subjective belief representation: source (Witnessed, Told, Inferred), confidence [0.0, 1.0], believed intent (Accidental vs Murder), supersession chain.
`minemesis/systems/simulation.py`	Simulation Host	Master simulation coordinator: executes monthly ticks, manages 4-tier lifecycle compaction (`compact_history`), tracks world survival.
`minemesis/systems/conflict.py`	Geopolitics	Evaluates feuds, food raids, succession crises, revenge murders, peace exhaustion, and the full multi-stage Truth Revelation pipeline.
`minemesis/religion/religion_engine.py`	Living Religion	Coordinates religion founding, holy site discovery, theological traits, moral virtues, holy pilgrimages, and cultural profile evolution.
`minemesis/religion/syncretism_engine.py`	Syncretism	Simulates cultural contact between cohabiting religions: doctrinal borrowing, syncretic faith emergence, heresy declaration, and schisms.
`minemesis/lore/lore_engine.py`	Literature Core	Coordinates monthly book authoring based on literacy rates and notable events; manages multi-repository library preservation.
`minemesis/lore/lore_generators.py`	Book Generators	9 procedural document generators producing rich multi-page books with deterministic on-demand lazy rendering (0 bytes heap during sim).
`minemesis/lore/lore_libraries.py`	Library Manager	Town archives, temple libraries, household shelves, scholar studies; archive fire/raid damage degradation; `unload_all_pages()`.
`viewer/app.py` & `viewer/model.py`	Story Viewer	Read-only Tkinter presentation layer with 9 tabs: interactive world map, causal DAG inspector, narrative threads, book reader, and Minecraft exporter.

Product Features & Technical Capabilities

Minemesis delivers a complete, cohesive simulation and presentation stack. Below is the technical breakdown of all major capabilities operating in the engine today.

1. Causal Graph & Deep Ancestry Tracing

Every event records explicit, validated causes. The engine can trace backward from any modern event (e.g. a peace treaty in Year 100) all the way back to the root forestry accident in Year 12 that triggered the original grudge.

- **Acyclic DAG Integrity:** Strictly zero circular causal loops.
- **Backward & Forward Inspection:** Query both "Why did this happen?" and "What were the consequences of this event?".
- **Deepest Conflict Path Analysis:** Automatically identifies the longest and most dramatic historical feud chains in the world.

2. Epistemic Psychology & Gossip Network

Agents experience reality subjectively. Rather than sharing a global knowledge base, each citizen constructs an individual belief ledger.

- **Three Belief Sources:** *Witnessed* (direct observation, high confidence), *Told* (hearsay via communication/trade routes), and *Inferred* (biased deductions based on character suspicion).
- **Belief Confidence & Decay:** Beliefs decay over time unless reinforced, enabling forgotten grievances and exhaustion peace.
- **Reappraisal upon Revelation:** When an eyewitness reveals the truth, agents update their beliefs, form emotional reappraisal memories, and formally reconcile.

3. Living Religion & Sacred Geography

Religion in Minemesis is not a static lore blurb; it is an active simulation agent driving human behavior and geopolitical alliances.

- **Theological Tenets & Commandments:** Emergent moral virtues, taboos, and ritual duties derived from settlement founder traits.
- **Sanctuaries & Pilgrimages:** Holy sites discovered at natural landmarks; citizens embark on holy pilgrimages, forming spiritual memories.
- **Syncretism, Heresy & Schisms:** Cohabiting faiths exchange doctrines; conservative priests declare heresies; ideological disputes cause sects to break away.

4. Procedural Literature Engine (9 Genres)

Elders, chiefs, priests, and travelers author authentic manuscripts from actual historical events using procedural grammatical templates (no LLMs).

- **9 Distinct Genres:** Family Histories, Personal Journals, Religious Scriptures, Myth Collections, Historical Chronicles, Local Folklore, Legal Records, Travelogues, and Biographies.
- **Subjective Author Perspective:** Books reflect the author's personal biases and beliefs rather than omniscient ground truth.
- **Library Repositories & War Damage:** Manuscripts stored across Town Archives and Temple Libraries; border raids burn libraries and corrupt parchment text.

5. Direct Minecraft Integration: Written Books & NBT Tags

Minemesis procedural books are directly compiled into standard **Minecraft Written Book NBT JSON** structures. Any book generated during centuries of simulation can be exported and spawned into vanilla Minecraft with a single command.

Minecraft NBT Written Book Output (Extract)

```
{
  "title": "Annals of Ironwood",
  "author": "Alden Thorne",
  "generation": 0,
  "pages": [
    "{\"text\":\"# Great Chronicle\\nPenned in Year 24.\\n\\nFormal trade route founded with Oakhaven...\"}"
  ],
  "lore": ["Genre: HistoricalChronicle", "Year: 24", "Words: 340"]
}
```

Vanilla /give Command Compatibility

The viewer generates ready-to-paste commands compatible with Minecraft 1.20+:

```
/give @p written_book{title:"Annals of Ironwood",author:"Alden Thorne",generation:0,pages:['"Year 24: Chief Alden ratified the Great Concord..."']} 1
```

✓ 100-page book pagination • ✓ Word-boundary title truncation • ✓ 16-character author cap

6. Minemesis Interactive Story Viewer (9 Tabs)

📜 Story & 🗺 Map Tabs

Interactive chronological event reader with color-coded categories, settlement filter, and full 2D map viewer displaying settlement coordinates, elevation, and terrain archetypes.

🔍 Why & ⚔ Threads Tabs

Visual causal DAG tree explorer explaining exactly why any chosen event happened, plus automated narrative thread extraction grouping feuds and dynasties into distinct story arcs.

📖 Books & 👤 People Tabs

Parchment-style Minecraft book reader with page navigation, paired with the Truth vs Belief inspector showing discrepancies between reality and character perception.

Computer Resource Usage & Empirical Benchmarks

Every performance metric reported below has been empirically measured and validated on live multi-century simulations. The transition from Milestone 0.5.3 to Milestone 0.5.4 achieved a massive **47× reduction in RAM consumption**.

1. Milestone 0.5.3 Baseline vs. Milestone 0.5.4 Achieved Results

Evaluation Metric	M0.5.3 Uncompacted	M0.5.4 Engineering Target	M0.5.4 Empirically Achieved	Improvement
Peak RAM (Year 100)	7,100 MB (7.1 GB)	≤ 2,000 MB (Hard limit ≤ 6 GB)	**64.1 – 154.0 MB (0.06 – 0.15 GB)**	47× REDUCTION
Peak RAM (Year 500)	Memory Thrashing (OOM)	≤ 6,000 MB	**269.7 MB (0.26 GB)**	STABLE EQUILIBRIUM
Active Epistemic Beliefs	2,850,000 beliefs	≥ 90% reduction	**16,481 – 46,438 beliefs**	98.74% REDUCTION
100-Year Average Runtime	180 – 300 seconds	≤ 180 seconds	**124.90 seconds**	30% FASTER
Demographic Conservation	Exact zero drift	$P = P_0 + B - D$	**Exact 0.00000% drift**	PERFECT CONSERVATION
Causal Grammar DAG Validity	0 illegal cycles	0 illegal cycles	**0 illegal cycles (100% Valid)**	100% CAUSAL PURITY

2. Multi-Seed 100-Year Benchmark Audit (Empirically Measured)

World Seed	Total Events	Living Population	Active Beliefs	Active Memories	Peak RAM (MB)	Runtime	Belief Compression
Seed 42	12,670	2,181	16,481	13,672	**64.11 MB**	28.18s	99.42% reduction
Seed 101	46,346	6,992	46,438	38,527	**154.04 MB**	175.66s	98.37% reduction
Seed 777	42,857	6,225	44,714	36,878	**147.38 MB**	170.86s	98.43% reduction

3. 4,000-Year Long-Term Scaling Projections

RAM Usage Projection across Eras

Under the uncompacted M0.5.3 architecture, RAM exploded quadratically with total accumulated characters and events ($O(E \times N)$). Under the 4-Tier Memory Lifecycle, dead citizens are compressed into `HistoricalCitizenArchive` and book pages are unloaded from heap memory, flattening growth to logarithmic.

- **100 Years:** 0.15 GB RAM (Measured)
- **500 Years:** 0.27 GB RAM (Measured)
- **1,000 Years:** ~0.45 GB RAM (Projected)
- **4,000 Years:** ~1.40 GB RAM (Projected • Well within 6 GB cap)

Demographic Carrying Capacity Dynamics

Population is stabilized by dynamic economic feedbacks: food carrying capacity, starvation during famines, border raid casualties, and disease/aging.

- **Initial Population:** ~120 – 250 founding citizens
- **Carrying Capacity:** 2,500 – 7,000 living citizens
- **Extinction Protection:** Soft carrying capacity guarantees thriving civilizations without runaway overpopulation or world extinction.

4. Hardware Requirements & Deployment Footprint

< 500 MB	1 Core	< 15 MB
TYPICAL WORKING SET RAM	CPU THREAD REQUIREMENT	DISK STORAGE PER WORLD JSON

Minemesis runs completely on standard consumer hardware (laptops, desktops, low-cost cloud VPS nodes). It does not require GPUs, neural accelerators, or persistent database servers.

Pros, Cons & Strategic Assessment

An honest assessment of our current strengths, engineering trade-offs, and remaining technical debt.

Technical Pros & Competitive Advantages

- **100% Deterministic & Auditable:** The identical seed pair produces the byte-identical history, event for event, on any operating system (Windows, Linux, macOS).
- **Zero Hallucination & Zero API Incurrence:** No monthly OpenAI/Anthropic token bills; no servers required; complete offline privacy.
- **Mathematically Verified:** 100% automated test coverage across 108 regression suites enforcing physical conservation and causal grammar.
- **True Narrative Depth:** Feuds are motivated by authentic misunderstandings; books reflect subjective human biases; religions split organically over doctrine.

Current Limitations & Technical Debt

- **Python Single-Thread CPU Bound:** Simulating 4,000 years continuously takes ~35–45 minutes on a single CPU thread. *Mitigation:* Multi-core multiprocessing across settlement regions, or ahead-of-time batch generation.
- **Topological Spatial Model:** Settlements currently exist on a 2D coordinate grid with Euclidean distance rather than full 3D voxel heightmaps. *Mitigation:* Milestone 1 world mapper translates 2D settlement coordinates directly into Minecraft world biomes.
- **GUI Framework:** Desktop viewer uses Tkinter for zero-dependency simplicity. While functional, it is not web-hosted or GPU-rendered.

Milestone 1 Roadmap: The Physical Minecraft Bridge

Milestone Stage	Deliverables	Impact on Player Experience
M1.1: Minecraft World Injector	Fabric/Forge Mod & Vanilla Datapack exporter translating Minemesis settlement coordinates into actual Minecraft chunk structures.	When a player creates a world, ancient settlements, abandoned ruins, and active villages appear exactly where history dictated.
M1.2: In-Game Physical Libraries	Town archives, lecterns, and chest inventories populated with physical Minecraft Written Books generated during simulation.	Players can explore village libraries and read real multi-generational histories, religious scriptures, and local gossip in-game.
M1.3: NPC Identity Mapping	Injecting living citizen names, lineages, professions, and personality traits into Minecraft villagers.	Villagers are no longer nameless mob dispensers; they are individuals with genealogies, grudges, and clan loyalties.
M1.4: Archeological Ruins & Tombs	Procedural placement of collapsed settlements, battle burial mounds, and hidden martyr shrines holding historical artifacts.	Exploring ruins reveals the tragic causes of settlement collapse (famine, raid devastation) documented in written inquests.

The Pitch to Backers & Supporters: Why Support Minemesis?

A Generational Paradigm Shift in Emergent Virtual Worlds

For twenty years, video games have treated procedural generation as a way to generate *geometry* (trees, caves, dungeons), while leaving *history and meaning* as hand-written, static backdrops. When developers have tried to generate dynamic history, they have hit the wall of memory explosion, nonsensical contradictions, or astronomical LLM API costs.

Minemesis proves that emergent, deep, coherent human history can be simulated algorithmically, deterministically, and cheaply on ordinary consumer hardware. We have conquered the 7.1 GB memory crisis, verified the causal grammar DAG, proven demographic invariance across centuries, synthesized 9 genres of procedural literature, and built direct compatibility with Minecraft.

With support for Milestone 1, Minemesis will deliver the first truly infinite, historically rich procedural universe in gaming history. We invite you to inspect our verified codebase, run our benchmarks, and partner with us to bring living history to millions of players worldwide.